



Trabalho prático 1

Fundamentos de sistemas operativos

ISEL – DEI – LEIM



Grupo 7:

Nº 523225 – Lucas Santos

Nº 52730 – Martim Gaspar

Nº 52731 – Miguel Nunes

Licenciatura em Engenharia Informática e Multimédia
Semestre de Inverno 2025/2026

Índice:

Introdução	3
Objetivo	3
UML	4
Teste da aplicação.....	6
GUI.....	6
Sincronização, Tarefas e Semáforos	8
Sincronização entre a GUI e os Movimentos Aleatórios.....	9
Código.....	10
Tarefa.....	10
Servidor.....	12
Movimentos aleatórios	15
Comando	18
Buffer Circular	19
Base de dados	21
Application.....	23
GUI	24
Conclusão	30
Referências	31

Índice de figuras:

Figura 1- UML do Projeto.....	4
Figura 2- GUI.....	6
Figura 3- Máquina de estados da classe tarefa.....	11
Figura 4- Máquina de estado dos movimentos aleatórios	17

Introdução

Este relatório descreve o desenvolvimento de uma aplicação em Java destinada ao controlo de um robô Lego EV3. O projeto implementa um sistema concorrente composto por vários módulos interligados, entre os quais se destacam a interface gráfica de utilizador (GUI), responsável pela interação com o operador, e a classe BaseDados, que centraliza o estado global da aplicação e as referências aos principais componentes.

O sistema utiliza mecanismos de sincronização baseados em semáforos para coordenar a comunicação entre tarefas produtoras e consumidoras, representadas respetivamente pelas classes MovimentosAleatorios e Servidor. Estas tarefas partilham um buffer circular que armazena os comandos a executar, permitindo um comportamento cooperativo e simultâneo entre os diferentes processos.

O movimento aleatório do robô é assim gerado e controlado de forma concorrente, demonstrando a integração de conceitos de programação multithread e sincronização. O desenvolvimento do projeto foi realizado ao longo de seis aulas práticas sob a orientação do professor Carlos Carvalho.

Objetivo

Os principais objetivos deste trabalho foram consolidar os conhecimentos na utilização do editor gráfico WindowBuilder, bem como aplicar conceitos de programação multitarefa e sincronização de processos em Java.

A interface gráfica foi desenvolvida para proporcionar uma interação intuitiva entre o utilizador e o sistema, permitindo o controlo visual do robô e a monitorização em tempo real dos seus parâmetros e estados. Através da GUI, o utilizador pode iniciar o modo de movimento aleatório, visualizar as ações geradas e acompanhar o comportamento do robô de forma clara e acessível.

O funcionamento do sistema baseia-se em duas tarefas principais que operam de forma concorrente: o produtor, representado pela classe MovimentosAleatorios, e o consumidor, implementado pela classe Servidor. A tarefa produtora é responsável por gerar comandos aleatórios — como avançar, curvar à direita ou à esquerda e parar — encapsulados em objetos da classe Comando. Estes comandos são então armazenados num BufferCircular, uma estrutura que atua como zona de partilha entre as tarefas.

O Servidor, por sua vez, funciona como consumidor desses comandos, retirando-os do buffer e executando as ações correspondentes sobre o robô (RobotLegoEV3). Esta interação é cuidadosamente controlada através de um mecanismo de sincronização baseado em semáforos, que assegura que o produtor e o consumidor acedem ao buffer de forma ordenada e segura, prevenindo condições de corrida e garantindo a integridade dos dados.

Esta arquitetura de comunicação cooperativa permitiu simular o comportamento dinâmico e aleatório do robô, integrando conceitos de programação concorrente, sincronização e design orientado a objetos.

Em suma, os objetivos propostos foram alcançados com sucesso, resultando numa aplicação funcional, robusta e eficiente, capaz de demonstrar de forma prática o uso de threads, semáforos e estruturas de dados partilhadas no controlo e simulação de um robô Lego EV3.

UML

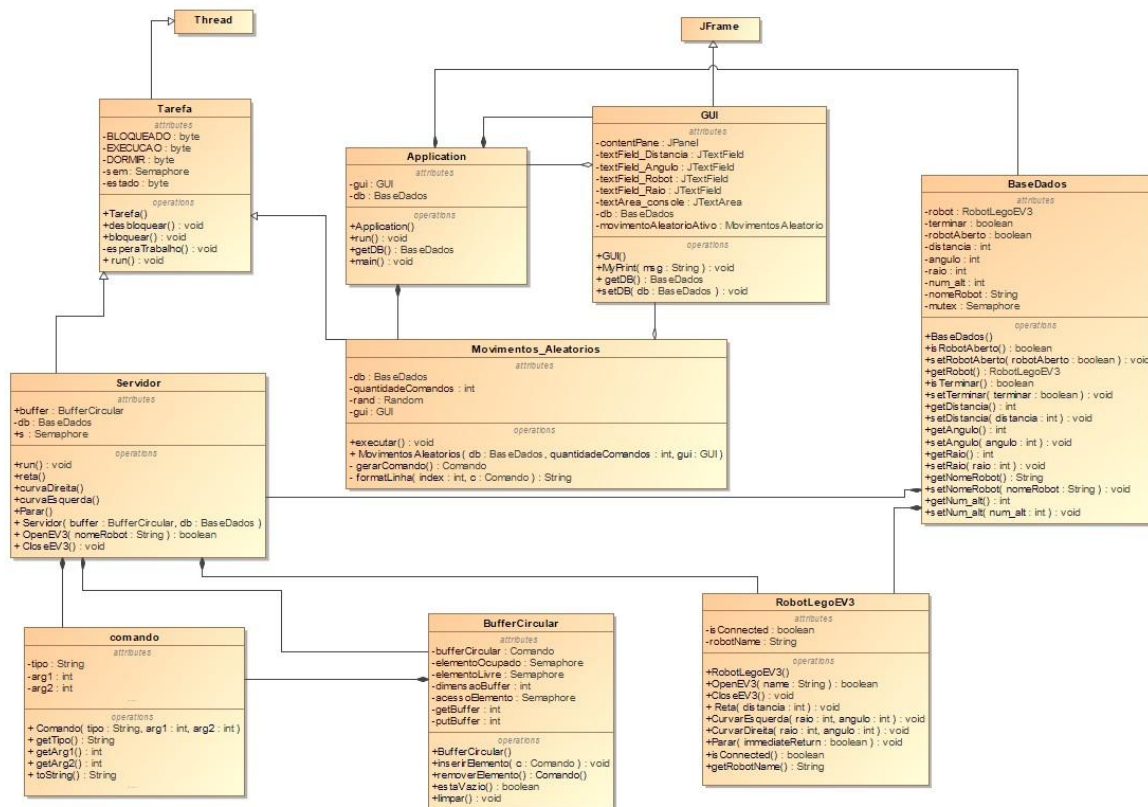


Figura 1- UML do Projeto

1. Tarefa:

Classe abstrata que representa o comportamento base de uma thread controlada por estados.

Implementa uma máquina de estados (BLOQUEADO, EXECUCAO, DORMIR), que define o ciclo de vida de cada tarefa.

Usa semáforos para coordenar a execução e permitir que as subclasses controlem quando iniciar, pausar ou dormir.

Serve como superclasse para todas as tarefas concorrentes do sistema, como Servidor e MovimentosAleatorios.

2. Servidor:

Subclasse de Tarefa que atua como consumidor no modelo produtor-consumidor. É responsável por ler comandos do buffer circular e executá-los no robô (RobotLegoEV3).

Cada comando contém parâmetros de movimento (distância, ângulo, raio), e o Servidor envia essas instruções de forma sequencial.

Ao consumir os comandos, também atualiza o estado na BaseDados e na interface gráfica.

3. MovimentosAleatorios:

Subclasse de Tarefa que funciona como produtor de comandos.

Gera comandos aleatórios para o robô, variando parâmetros como distância, ângulo e raio.

Esses comandos são encapsulados em objetos Comando e inseridos no buffer circular, respeitando as regras de sincronização impostas pelos semáforos.

A sua execução contínua permite ao sistema simular movimentos imprevisíveis e autônomos do robô.

4. BufferCircular:

Classe que implementa o buffer de comunicação entre o produtor (MovimentosAleatorios) e o consumidor (Servidor).

Utiliza semáforos de sincronização (cheio e vazio) e exclusão mútua para controlar o acesso simultâneo das threads.

Garante que o produtor apenas insira comandos quando houver espaço e que o consumidor só retire quando houver comandos disponíveis.

É essencial para evitar condições de corrida e deadlocks no sistema.

5. BaseDados:

Classe que centraliza e partilha o estado global da aplicação.

Armazena referências para o robô, o buffer, as tarefas e as variáveis de controlo, permitindo a comunicação entre os diferentes componentes.

Serve também como ponto de acesso comum para a GUI, possibilitando que o utilizador visualize e controle o estado do sistema.

Funciona como uma espécie de “núcleo” de dados partilhados.

6. Comando:

Classe simples (modelo de dados) que representa um conjunto de instruções para o robô executar.

Contém parâmetros como distância, ângulo e raio, que determinam o tipo e a intensidade do movimento.

É usada tanto pelo produtor (que cria novos comandos) quanto pelo consumidor (que interpreta e executa).

7. RobotLegoEV3:

Classe responsável pelo comportamento do robô Lego EV3.

Recebe comandos do Servidor e executa ações correspondentes (movimento em frente, curvas, paragens, etc.).

8. GUI:

Classe que constrói a interface gráfica do utilizador com a biblioteca Swing (e projetada com o WindowBuilder).

Permite iniciar ou parar o sistema, visualizar o estado das tarefas, e acompanhar os parâmetros de execução dos comandos.

Comunica-se com a BaseDados para aceder aos dados das tarefas e atualizar dinamicamente as informações apresentadas.

Teste da aplicação

GUI

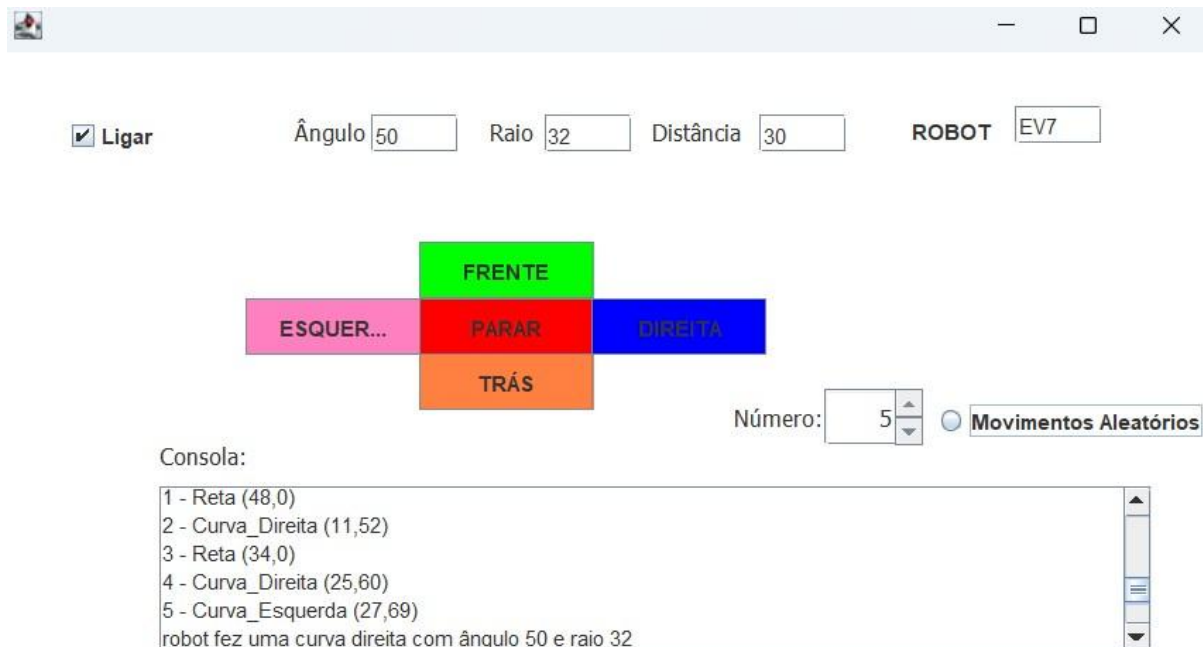


Figura 2- GUI

A **GUI (Graphical User Interface)** constitui a parte visual e interativa da aplicação, permitindo a **comunicação direta entre o utilizador e o programa em Java**. Foi desenvolvida com recurso à biblioteca **Swing** e ao editor **WindowBuilder**, proporcionando um ambiente intuitivo e funcional para o controlo do robô LEGO EV3.

A interface é composta por vários elementos principais:

- Um **checkbox “Ligar”**, responsável por estabelecer e encerrar a ligação entre a aplicação e o robô físico. Quando ativado, o robô é inicializado e o sistema desbloqueia a execução das tarefas; quando desativado, a ligação é encerrada e as threads são bloqueadas.
- **Campos de texto (TextFields)** para **Ângulo**, **Raio** e **Distância**, onde o utilizador pode definir os valores que serão utilizados como parâmetros nos comandos de movimento enviados ao robô. Estes valores determinam o comportamento físico do robô durante a execução das ações.

- **Botões de controlo principais:**
 - **Frente**, para fazer o robô avançar; o **Trás**, para realizar marcha-atrás;
 - **Curva Direita e Curva Esquerda**, que executam rotações com base no ângulo e raio definidos;
 - **Parar**, que interrompe imediatamente o movimento e limpa o buffer de comandos, assegurando que o robô pare em segurança.
- Uma secção dedicada ao **movimento aleatório**, que permite definir o **número de comandos aleatórios** a gerar e enviar para o buffer. Nesta área existe também um **botão de ativação**, que inicia ou interrompe a tarefa de MovimentosAleatorios, responsável por gerar esses comandos de forma automática.
- Por fim, existe um **campo de consola (TextArea)**, onde são apresentadas mensagens informativas e registos das ações executadas, indicando em tempo real os comandos enviados e o estado atual do robô.

A GUI desempenha, portanto, um papel essencial no projeto, servindo como **ponte entre o utilizador e as tarefas internas do sistema**. Através dela é possível controlar, monitorizar e interagir com o robô de forma simples, visual e segura, refletindo a integração prática entre a interface gráfica e a programação multitarefa implementada em Java.

Sincronização, Tarefas e Semáforos

De acordo com Silberschatz, Galvin e Gagne em *Operating System Concepts* (8ª edição), a sincronização é um dos conceitos fundamentais da programação concorrente, utilizada para coordenar a execução de múltiplas tarefas (ou threads) que partilham recursos comuns. Cada tarefa representa uma unidade independente de execução dentro de um processo, possuindo o seu próprio fluxo de instruções, mas partilhando memória e variáveis globais com as restantes. A execução simultânea destas tarefas, sem qualquer controlo, pode originar conflitos de acesso e comportamentos indeterminados, motivo pelo qual se recorre a mecanismos de sincronização.

Um dos mecanismos mais clássicos e eficazes é o semáforo, introduzido por Dijkstra, que consiste numa variável controlada por duas operações atómicas: `wait()` e `signal()`. Os semáforos permitem regular o acesso a recursos partilhados e garantir exclusão mútua, ou seja, que apenas uma tarefa aceda de cada vez à secção crítica do código. Existem dois tipos principais: o semáforo binário, utilizado para garantir exclusão mútua, e o semáforo de contagem, usado para controlar o número de recursos disponíveis.

Outro conceito importante abordado no livro é o problema produtor-consumidor, também conhecido como o problema do buffer limitado (*bounded-buffer problem*). Neste modelo, uma tarefa produtora gera dados e coloca-os num buffer, enquanto uma tarefa consumidora retira esses dados para processamento. Sem sincronização adequada, o produtor pode tentar inserir dados num buffer cheio, ou o consumidor pode tentar ler de um buffer vazio, levando a falhas de execução.

No contexto do nosso projeto, estes conceitos foram aplicados de forma prática no controlo do robô LEGO EV3. A aplicação desenvolvida em Java utiliza tarefas (threads) que trabalham de forma concorrente: o produtor (*MovimentosAleatorios*) gera comandos de movimento, enquanto o consumidor (*Servidor*) lê e executa esses comandos no robô. A comunicação entre as tarefas é feita através de um buffer circular, que implementa precisamente o modelo produtor-consumidor, garantindo o fluxo ordenado de instruções. A sincronização entre as tarefas foi implementada com semáforos, de modo a evitar acessos simultâneos ao robô e garantir que apenas uma tarefa controla os seus motores num determinado instante. A classe *Tarefa* foi fundamental nesta arquitetura, pois gere os estados das threads (*BLOQUEADO*, *EXECUCAO*, *DORMIR*) e assegura o controlo do ciclo de vida de cada uma.

Com esta abordagem, o sistema consegue executar movimentos coordenados e previsíveis, mesmo com múltiplas tarefas a correr em paralelo. Assim, o trabalho desenvolvido demonstra de forma clara e prática os princípios teóricos de sincronização, comunicação entre tarefas e utilização de semáforos apresentados no livro *Operating System Concepts*, aplicados a um caso real do nosso projecto.

Sincronização entre a GUI e os Movimentos Aleatórios

No projeto desenvolvido, a sincronização entre a interface gráfica (GUI) e a tarefa `MovimentosAleatorios` foi implementada de forma a garantir que ambas as partes do sistema interagem com o robô LEGO EV3 sem provocar conflitos nem acessos simultâneos indevidos.

A aplicação utiliza o conceito de tarefas (threads) para executar diferentes funções em paralelo: a GUI trata das ações do utilizador e da interação visual, enquanto a tarefa `MovimentosAleatorios` funciona em segundo plano, onde gera e envia comandos aleatórios para o robô através do buffer circular.

Para assegurar o controlo coordenado entre estas duas entidades, utiliza-se um semáforo partilhado (instância da classe `Semaphore`) declarado na classe `Servidor`. Tanto a GUI como a tarefa `MovimentosAleatorios` conhecem e acedem a este mesmo semáforo, o que lhes permite sincronizar o acesso aos recursos comuns — nomeadamente o robô físico e o buffer de comandos.

Quando o utilizador pressiona um botão na GUI (por exemplo, “Frente” ou “Curva Direita”), o programa adquire o semáforo (`s.acquire()`), envia o comando correspondente para o robô e liberta-o de seguida (`s.release()`). Durante esse intervalo, a tarefa `MovimentosAleatorios` fica impedida de inserir novos comandos, o que garante que o movimento manual tem prioridade e não sofre interferências.

De forma semelhante, quando a tarefa `MovimentosAleatorios` está ativa, adquire o mesmo semáforo antes de colocar comandos aleatórios no buffer. Desta forma, impede que a GUI tente enviar outro comando ao mesmo tempo, evitando instruções contraditórias.

Este mecanismo assegura exclusão mútua, ou seja, apenas uma das threads (GUI ou `MovimentosAleatorios`) pode controlar o robô num determinado instante. O resultado é uma sincronização eficaz entre a interação do utilizador e a execução automática dos movimentos, prevenindo comportamentos imprevistos e garantindo a segurança e previsibilidade das ações do robô.

Em conclusão, o semáforo partilhado atua como o elo de ligação entre as duas partes do sistema — a interface de controlo humano (GUI) e a tarefa automática (`MovimentosAleatorios`). Esta coordenação constitui um exemplo prático da aplicação dos princípios de sincronização e exclusão mútua estudados em programação concorrente, demonstrando a correta cooperação entre tarefas num sistema multitarefa real.

Código

Tarefa

```
import java.util.concurrent.Semaphore;

public abstract class Tarefa extends Thread{
    private final byte BLOQUEADO = 0, EXECUCAO = 1, DORMIR = 2;

    private Semaphore sem;
    private byte estado;

    public Tarefa() {
        estado = BLOQUEADO;
        sem = new Semaphore(0);
    }

    public void desbloquear() {
        estado = EXECUCAO;
        sem.release();
    }

    public void bloquear() {
        estado = BLOQUEADO;
        try {
            sem.acquire();
        } catch (InterruptedException e) {e.printStackTrace();}
    }

    private void esperaTrabalho() {
        try {
            sem.acquire();
        } catch (InterruptedException e) {e.printStackTrace();}
    }

    protected abstract void runing();

    protected void dormir() {
        try {
            Thread.sleep((long) Math.random() * 1000);
        } catch (InterruptedException e) {e.printStackTrace();}
    }
}
```

```

public void run() {
    while (true) {
        switch (estado) {
            case BLOQUEADO:
                esperaTrabalho();
                break;
            case EXECUCAO:
                runing();
                if (estado == EXECUCAO)
                    estado = DORMIR;
                break;
            case DORMIR:
                dormir();
                if (estado == DORMIR)
                    estado = EXECUCAO;
                break;
        }
    }
}

```



Figura 3- Máquina de estados da classe tarefa

Servidor

```
/**Thread consumidor
 *
 * Executa comandos do buffer e simula o tempo real de movimento
 *
 * *****/

import java.util.Objects;
import java.util.concurrent.Semaphore;

public class Servidor extends Tarefa {
    public final BufferCircular buffer;
    private final BaseDados db;
    private final RobotLegoEV3Simula robot;
    public final Semaphore s = new Semaphore(1); //sincronização com a gui

    public Servidor(BufferCircular buffer, BaseDados db) {
        this.buffer = Objects.requireNonNull(buffer);
        this.db = Objects.requireNonNull(db);
        this.robot = new RobotLegoEV3Simula();
        this.start();
    }

    private final double vel = 0.02;
    private final int comunicacao = 100;

    public void Reta(int distancia) { buffer.inserirElemento(new Comando("RETA",
    distancia, 0)); }
    public void CurvarDireita(int raio, int angulo) { buffer.inserirElemento(new
    Comando("CURVA_DIREITA", raio, angulo)); }
    public void CurvarEsquerda(int raio, int angulo) { buffer.inserirElemento(new
    Comando("CURVA_ESQUERDA", raio, angulo)); }

    public void Parar(boolean forcar) {
        if (forcar) {
            System.out.println("[Servidor] Parada forçada — limpando buffer e parando
    robô.");
            buffer.limpar();    // esvazia todos os comandos pendentes
            robot.Parar(true);  // para imediatamente o robô
        } else {
            buffer.inserirElemento(new Comando("PARAR", 0, 0));
        }
    }
}
```

```

}

public synchronized boolean OpenEV3(String nomeRobot) {
    boolean ok = robot.OpenEV3(nomeRobot);
    if (ok) {
        db.setRobotAberto(true);
        desbloquear(); // acorda a thread se estava bloqueada
    } else {
        db.setRobotAberto(false);
    }
    return ok;
}

```

```

public synchronized void CloseEV3() {
    robot.CloseEV3();
    db.setRobotAberto(false);
    bloquear(); // pausa a thread até reabrir
}

```

```

@Override
protected void runing() {
    try {
        Comando c = buffer.removerElemento();
        if (c != null) executarComandoNoRobot(c);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

private int tempoReta(int d) {
    // d está em cm, vel em cm/ms → resultado em ms
    return (int) ((d / vel) + comunicacao);
}

```

```

private int tempoCurva(int r, int a) {
    double rad = 2 * Math.PI * r * (a / 360.0);
    return (int) ((rad / vel) + comunicacao);
}

```

```

private int tempoParar() {
    return comunicacao;
}

```

```

private void executarComandoNoRobot(Comando c) {
    if (c == null) return;

    switch (c.getTipo()) {
        case "RETA":

```

```

        robot.Reta(c.getArg1());
        sleepTempo(tempoReta(c.getArg1()));
        break;

    case "CURVA_DIREITA":
        robot.CurvarDireita(c.getArg1(), c.getArg2());
        sleepTempo(tempoCurva(c.getArg1(), c.getArg2()));
        break;

    case "CURVA_ESQUERDA":
        robot.CurvarEsquerda(c.getArg1(), c.getArg2());
        sleepTempo(tempoCurva(c.getArg1(), c.getArg2()));
        break;

    case "PARAR":
        robot.Parar(false);
        sleepTempo(tempoParar());
        break;

    default:
        System.out.println("[Servidor] Comando desconhecido: " + c.getTipo());
        break;
    }
}

private void sleepTempo(int ms) {
    try { Thread.sleep(ms); } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); }
    }
}

```

Movimentos aleatórios

```
/******
 *
 * Thread produtora
 *
 * Gera comandos e coloca no buffer
 *
 * *****/
import java.util.Random;

public class MovimentosAleatorios extends Tarefa {
    private final BaseDados db;
    private final int quantidadeComandos;
    private final Random rand;
    private final GUI gui;

    public MovimentosAleatorios(BaseDados db, int quantidadeComandos, GUI gui) {
        this.db = db;
        this.quantidadeComandos = Math.max(1, quantidadeComandos);
        this.rand = new Random();
        this.gui = gui;
    }

    private Comando gerarComando() {
        int tipo = rand.nextInt(3);
        switch (tipo) {
            case 0:
                return new Comando("RETA", 10 + rand.nextInt(41), 0);
            case 1:
                return new Comando("CURVA_DIREITA", 10 + rand.nextInt(21), 20 +
rand.nextInt(71));
            default:
                return new Comando("CURVA_ESQUERDA", 10 + rand.nextInt(21), 20 +
rand.nextInt(71));
        }
    }

    private String formatLinha(int index, Comando c) {
        String nome;
        switch (c.getTipo()) {
            case "RETA":
                nome = "Reta"; break;
            case "CURVA_DIREITA":
                nome = "Curva_Direita"; break;
            case "CURVA_ESQUERDA":
                nome = "Curva_Esquerda"; break;
            default:
                nome = c.getTipo(); break;
        }
    }
}
```

```

    }
    return index + " - " + nome + " (" + c.getArg1() + "," + c.getArg2() + ")";
}

@Override
protected void runing() {
    if (db.getServidor() == null) {
        try { Thread.sleep(500); } catch (InterruptedException e) { return; }
        return; // espera servidor pronto
    }

    for (int i = 0; i < quantidadeComandos; i++) {
        Comando c = gerarComando();
        db.getServidor().buffer.inserirElemento(c);
        if (gui != null) gui.MyPrint(formatLinha(i + 1, c));

        try {
            Thread.sleep(150);
        } catch (InterruptedException e) {
            return;
        }
    }

    // finaliza com comando PARAR
    Comando parar = new Comando("PARAR", 0, 0);
    db.getServidor().buffer.inserirElemento(parar);
    if (gui != null) gui.MyPrint("Parar(false)");
}
}

```

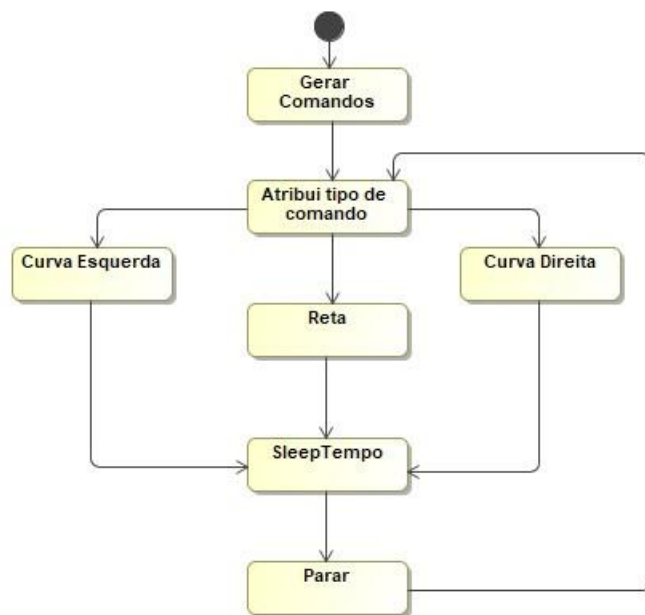


Figura 4- Máquina de estado dos movimentos aleatórios

A classe `MovimentosAleatorios` representa uma tarefa automática responsável por gerar e enviar comandos aleatórios para o robô LEGO EV3. Esta tarefa é executada numa thread separada e funciona em paralelo com a interface gráfica (GUI). O seu objetivo é simular um comportamento autónomo do robô, sem a necessidade de intervenção direta do utilizador.

Quando é ativada através da GUI, a tarefa cria uma sequência de comandos que podem ser do tipo Reta, Curva Direita ou Curva Esquerda, escolhidos de forma aleatória. Cada comando é colocado no buffer circular, de onde o Servidor os lê e executa no robô. No final da sequência, é enviado o comando Parar, garantindo que o robô termina o movimento de forma controlada.

O diagrama apresentado ilustra o funcionamento desta classe. O processo começa na geração dos comandos, onde é atribuído um tipo de movimento. Dependendo do resultado aleatório, o sistema escolhe entre Curva Esquerda, Reta ou Curva Direita. Após o envio de cada comando, o programa entra num curto período de espera (`SleepTempo`) para simular o tempo real de execução do movimento. No final, é emitido o comando Parar, encerrando o ciclo antes de poder recomeçar uma nova sequência.

Comando

```
public class Comando {
    private final String tipo;
    private final int arg1;
    private final int arg2;

    public Comando(String tipo, int arg1, int arg2) {
        this.tipo = tipo;
        this.arg1 = arg1;
        this.arg2 = arg2;
    }

    public String getTipo() {
        return tipo;
    }

    public int getArg1() {
        return arg1;
    }

    public int getArg2() {
        return arg2;
    }

    @Override
    public String toString() {
        return String.format("Comando[%s, %d, %d]", tipo, arg1, arg2);
    }
}
```

Buffer Circular

```
import java.util.concurrent.Semaphore;
import java.util.Random;

/* ===== Buffer Circular ===== */
public class BufferCircular {
    private final int dimensaoBuffer = 16;
    private final Comando[] bufferCircular;
    private int putBuffer, getBuffer;

    private final Semaphore elementosLivres;
    private final Semaphore elementosOcupados;
    private final Semaphore acessoElemento;

    public BufferCircular() {
        bufferCircular = new Comando[dimensaoBuffer];
        putBuffer = 0;
        getBuffer = 0;
        elementosLivres = new Semaphore(dimensaoBuffer);
        elementosOcupados = new Semaphore(0);
        acessoElemento = new Semaphore(1);
    }

    public void inserirElemento(Comando c) {
        try {
            elementosLivres.acquire();
            acessoElemento.acquire();
            bufferCircular[putBuffer] = c;
            putBuffer = (putBuffer + 1) % dimensaoBuffer;
            acessoElemento.release();
            elementosOcupados.release();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```

public Comando removerElemento() {
    Comando c = null;
    try {
        elementosOcupados.acquire();
        acessoElemento.acquire();
        c = bufferCircular[getBuffer];
        bufferCircular[getBuffer] = null;
        getBuffer = (getBuffer + 1) % dimensaoBuffer;
        acessoElemento.release();
        elementosLivres.release();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    return c;
}

public boolean estaVazio() {
    return elementosOcupados.availablePermits() == 0;
}

public void limpar() {
    try {
        elementosOcupados.drainPermits();
        acessoElemento.acquire();
        putBuffer = getBuffer;
        elementosLivres.release(dimensaoBuffer - elementosLivres.availablePermits());
        acessoElemento.release();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}

```

Base de dados

```
public class BaseDados {
    private boolean terminar;
    private boolean robotAberto;
    private String nomeRobot;

    private final BufferCircular buffer;
    private final Servidor servidor; // Servidor único e final

    private int ultimaDistancia;
    private int ultimoAngulo;
    private int ultimoRaio;

    public BaseDados() {
        this.terminar = false;
        this.nomeRobot = "EV7";
        this.robotAberto = false;
        this.buffer = new BufferCircular();
        this.servidor = new Servidor(buffer, this);
    }

    public Servidor getServidor() {
        return servidor;
    }

    public BufferCircular getBuffer() {
        return buffer;
    }

    public boolean isRobotAberto() {
        return robotAberto;
    }

    public void setRobotAberto(boolean robotAberto) {
        this.robotAberto = robotAberto;
    }

    public boolean isTerminar() {
        return terminar;
    }

    public void setTerminar(boolean terminar) {
        this.terminar = terminar;
    }

    public String getNomeRobot() {
        return nomeRobot;
    }
}
```

```

    }

    public void setNomeRobot(String nomeRobot) {
        this.nomeRobot = nomeRobot;
    }

    public int getUltimaDistancia() {
        return ultimaDistancia;
    }

    public void setUltimaDistancia(int ultimaDistancia) {
        this.ultimaDistancia = ultimaDistancia;
    }

    public int getUltimoAngulo() {
        return ultimoAngulo;
    }

    public void setUltimoAngulo(int ultimoAngulo) {
        this.ultimoAngulo = ultimoAngulo;
    }

    public int getUltimoRaio() {
        return ultimoRaio;
    }

    public void setUltimoRaio(int ultimoRaio) {
        this.ultimoRaio = ultimoRaio;
    }
}

```

Application

```
public class Application {
    private GUI gui;
    private BaseDados db;

    public Application() {
        db = new BaseDados(); // BaseDados já cria o buffer e servidor
        gui = new GUI(this);
        gui.setVisible(true);
    }

    public BaseDados getDB() {
        return db;
    }

    public void run() {
        System.out.println("A aplicação começou!!");

        while (!db.isTerminar()) {
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        // Finalização
        db.getServidor().interrupt();
        System.out.println("A aplicação terminou!");
    }

    public static void main(String[] args) {
        Application app = new Application();
        app.run();
    }
}
```

GUI

```
import java.awt.EventQueue;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.beans.Beans;

import javax.swing.*;
import javax.swing.border.EmptyBorder;

import java.awt.Color;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class GUI extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;

    private JTextField textField_Distancia;
    private JTextField textField_Angulo;
    private JTextField textField_Robot;
    private JTextField textField_Raio;
    private JTextArea textArea_console;

    private BaseDados db;
    private MovimentosAleatorios movimentoAleatorioAtivo = null;

    public void MyPrint(String msg) {
        SwingUtilities.invokeLater(() -> textArea_console.append(msg + "\n"));
    }

    public GUI(Application app) {
        db = app.getDB();

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 702, 420);

        contentPane = new JPanel();
        contentPane.setBackground(Color.WHITE);
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);
        contentPane.setLayout(null);

        // ----- BOTÕES -----
        JButton btnFrente = new JButton("FRENTE");
        btnFrente.setBackground(Color.GREEN);
        btnFrente.setBounds(237, 108, 99, 32);
```

```

btnFrente.addActionListener(e -> {
    try {
        db.getServidor().s.acquire();
        db.getServidor().Reta(db.getUltimaDistancia());
        db.getServidor().Parar(false);
    } catch (InterruptedException e1) {
        Thread.currentThread().interrupt();
    } finally {
        db.getServidor().s.release();
    }
    MyPrint("fiz uma reta com " + db.getUltimaDistancia());
});
contentPane.add(btnFrente);

JButton btnTras = new JButton("TRÁS");
btnTras.setBackground(new Color(255, 128, 64));
btnTras.setBounds(237, 171, 99, 32);
btnTras.addActionListener(e -> {
    try {
        db.getServidor().s.acquire();
        db.getServidor().Reta(- db.getUltimaDistancia());
        db.getServidor().Parar(false);
    } catch (InterruptedException e1) {
        Thread.currentThread().interrupt();
    } finally {
        db.getServidor().s.release();
    }
    MyPrint("fiz uma marcha trás com " + db.getUltimaDistancia());
});
contentPane.add(btnTras);

JButton btnParar = new JButton("PARAR");
btnParar.setBackground(Color.RED);
btnParar.setBounds(237, 140, 99, 32);
btnParar.addActionListener(e -> {
    if (movimentoAleatorioAtivo != null) {
        movimentoAleatorioAtivo.bloquear();
        movimentoAleatorioAtivo = null;
        MyPrint("Movimentos aleatórios bloqueados.");
    }

    db.getServidor().Parar(true); // agora é seguro
    MyPrint("Robô parado (forçado), buffer limpo, servidor continua ativo.");
});

contentPane.add(btnParar);
JButton btnDir = new JButton("DIREITA");
btnDir.setBackground(Color.BLUE);
btnDir.setBounds(335, 140, 99, 32);

```

```

btnDir.addActionListener(e -> {
    try {
        db.getServidor().s.acquire();
        db.getServidor().CurvarDireita(db.getUltimoRaio(), db.getUltimoAngulo());
        db.getServidor().Parar(false);
    } catch (InterruptedException e1) {
        Thread.currentThread().interrupt();
    } finally {
        db.getServidor().s.release();
    }
    MyPrint("robot fez uma curva direita com ângulo " + db.getUltimoAngulo() + " e
raio " + db.getUltimoRaio());
});
contentPane.add(btnDir);

JButton btnEsq = new JButton("ESQUERDA");
btnEsq.setBackground(new Color(255, 128, 192));
btnEsq.setBounds(139, 140, 99, 32);
btnEsq.addActionListener(e -> {
    try {
        db.getServidor().s.acquire();
        db.getServidor().CurvarEsquerda(db.getUltimoRaio(), db.getUltimoAngulo());
        db.getServidor().Parar(false);
    } catch (InterruptedException e1) {
        Thread.currentThread().interrupt();
    } finally {
        db.getServidor().s.release();
    }
    MyPrint("robot fez uma curva esquerda com ângulo " + db.getUltimoAngulo() +
" e raio " + db.getUltimoRaio());
});
contentPane.add(btnEsq);

// ----- CHECKBOX LIGAR -----
JCheckBox checkLigar = new JCheckBox("Ligar");
checkLigar.setBounds(36, 36, 97, 23);
checkLigar.setBackground(Color.WHITE);
checkLigar.setSelected(db.isRobotAberto());
checkLigar.addActionListener(e -> {
    if (db.isRobotAberto()) {
        db.getServidor().CloseEV3();
        db.setRobotAberto(false);
        textField_Robot.setText("");
        MyPrint("Servidor bloqueado e conexão encerrada.");
    } else {
        boolean aberto = db.getServidor().OpenEV3(db.getNomeRobot());
        db.setRobotAberto(aberto);
        if (aberto) {
            db.getServidor().desbloquear(); // ☒ apenas desbloqueia, sem start

```

```

        textField_Robot.setText(db.getNomeRobot());
        MyPrint("Servidor desbloqueado e ativo.");
    } else {
        MyPrint("Falha ao abrir conexão com o robô.");
    }
}
checkLigar.setSelected(db.isRobotAberto());
});

contentPane.add(checkLigar);

// ----- CAMPOS -----
// Distância
JLabel lblDistancia = new JLabel("Distância");
lblDistancia.setFont(new Font("Tahoma", Font.PLAIN, 13));
lblDistancia.setBounds(369, 30, 70, 32);
contentPane.add(lblDistancia);

textField_Distancia = new JTextField("" + db.getUltimaDistancia());
textField_Distancia.setBounds(430, 36, 50, 22);
textField_Distancia.setColumns(10);
textField_Distancia.addActionListener(e -> {
    db.setUltimaDistancia(Integer.parseInt(textField_Distancia.getText()));
    MyPrint("a distancia foi alterada para : " + db.getUltimaDistancia());
});
contentPane.add(textField_Distancia);

// Ângulo
JLabel lblAngulo = new JLabel("Ângulo");
lblAngulo.setFont(new Font("Tahoma", Font.PLAIN, 13));
lblAngulo.setBounds(166, 30, 70, 32);
contentPane.add(lblAngulo);

textField_Angulo = new JTextField("" + db.getUltimoAngulo());
textField_Angulo.setColumns(10);
textField_Angulo.setBounds(210, 36, 50, 22);
textField_Angulo.addActionListener(e -> {
    db.setUltimoAngulo(Integer.parseInt(textField_Angulo.getText()));
    MyPrint("o ângulo foi alterado para : " + db.getUltimoAngulo());
});
contentPane.add(textField_Angulo);

// Raio
JLabel lblRaio = new JLabel("Raio");
lblRaio.setFont(new Font("Tahoma", Font.PLAIN, 13));
lblRaio.setBounds(277, 30, 34, 32);
contentPane.add(lblRaio);

textField_Raio = new JTextField("" + db.getUltimoRaio());

```

```

textField_Raio.setColumns(10);
textField_Raio.setBounds(308, 36, 50, 22);
textField_Raio.addActionListener(e -> {
    db.setUltimoRaio(Integer.parseInt(textField_Raio.getText()));
    MyPrint("o raio foi alterado para : " + db.getUltimoRaio());
});
contentPane.add(textField_Raio);

// Robot
JLabel lblRobot = new JLabel("ROBOT");
lblRobot.setBounds(516, 37, 56, 16);
contentPane.add(lblRobot);

textField_Robot = new JTextField();
textField_Robot.setColumns(10);
textField_Robot.setBounds(575, 31, 50, 22);
contentPane.add(textField_Robot);

// ----- MOVIMENTOS ALEATÓRIOS -----
JLabel lblNumero = new JLabel("Número:");
lblNumero.setFont(new Font("Tahoma", Font.PLAIN, 13));
lblNumero.setBounds(415, 195, 65, 25);
contentPane.add(lblNumero);

JSpinner spinner = new JSpinner();
spinner.setFont(new Font("Tahoma", Font.PLAIN, 13));
spinner.setBounds(467, 191, 56, 32);
contentPane.add(spinner);

JRadioButton rdbtnMovAlt = new JRadioButton("Movimentos Aleatórios");
rdbtnMovAlt.setBackground(Color.WHITE);
rdbtnMovAlt.setBounds(529, 198, 180, 23);
contentPane.add(rdbtnMovAlt);

rdbtnMovAlt.addActionListener(e -> {
    if (!rdbtnMovAlt.isSelected()) {
        if (movimentoAleatorioAtivo != null) {
            movimentoAleatorioAtivo.bloquear();
            movimentoAleatorioAtivo = null;
            MyPrint("Movimentos aleatórios bloqueados.");
        }

        db.getServidor().Parar(true); // para completamente e limpa buffer
        MyPrint("Parada forçada após desativar movimentos aleatórios.");
        return;
    }
});

int qtd = (int) spinner.getValue();

```

```

        movimentoAleatorioAtivo = new MovimentosAleatorios(db, qtd, this);
        movimentoAleatorioAtivo.start();    // inicia uma única vez
        movimentoAleatorioAtivo.desbloquear(); // começa o loop de runing()
        MyPrint("Gerando " + qtd + " movimentos aleatórios...");
    });
    // ----- CONSOLE -----
    JScrollPane scrollPane = new JScrollPane();
    scrollPane.setBounds(90, 246, 563, 95);
    contentPane.add(scrollPane);

    textArea_console = new JTextArea();
    scrollPane.setViewportView(textArea_console);

    JLabel lblConsole = new JLabel("Consola:");
    lblConsole.setFont(new Font("Tahoma", Font.PLAIN, 13));
    lblConsole.setBounds(90, 221, 82, 16);
    contentPane.add(lblConsole);

    // ----- WINDOW CLOSE -----
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            if (db.isRobotAberto()) {
                db.getServidor().CloseEV3();
            }

            db.setTerminar(true);
        }
    });
}

public BaseDados getDB() {
    return db;
}

public void setDB(BaseDados db) {
    this.db = db;
}
}

```

Conclusão

Neste trabalho prático, consolidámos os conhecimentos de programação multitarefa em Java, aplicando mecanismos de sincronização com semáforos e comunicação entre threads para o controlo real de um robô LEGO EV3.

O desenvolvimento da aplicação envolveu a integração de vários componentes — como a interface gráfica (GUI), o buffer circular, a classe Servidor, a classe MovimentosAleatorios e a classe BaseDados — que, em conjunto, implementam uma arquitetura produtora-consumidora para a execução ordenada de comandos no robô.

Ao longo das aulas práticas, fomos testando cada parte da aplicação diretamente no robô, verificando a resposta física aos comandos “Frente”, “Trás”, “Esquerda”, “Direita” e “Parar”. Essa abordagem permitiu validar o correto funcionamento do sistema em tempo real, bem como observar a importância da sincronização entre tarefas para evitar conflitos no controlo do robô.

Na última aula prática, atingimos aproximadamente 80% da implementação completa — com a interface, comunicação e execução de comandos já operacionais. As partes finais foram concluídas em casa, nomeadamente a implementação completa da classe Tarefa, responsável pela gestão dos estados de execução das threads, e a correção do botão “Movimentos Aleatórios”, que agora utiliza corretamente o método Parar(true), permitindo interromper os movimentos de forma segura e controlada.

Com estas melhorias, a aplicação ficou totalmente funcional, executando o ciclo completo de geração, envio e execução de comandos no robô físico, com sincronização eficiente entre produtor e consumidor.

Em conclusão, consideramos que todos os objetivos foram alcançados com sucesso, resultando numa aplicação robusta, estável e funcional, que demonstra na prática os conceitos de multitarefa, sincronização e controlo cooperativo de tarefas em sistemas reais.

Referências

- PAIS, Jorge. *Fundamentos de Sistemas Operativos*. Versão 2, 2025–2026. Instituto Superior de Engenharia de Lisboa (ISEL).
- SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. *Operating System Concepts*. 8.^a edição. John Wiley & Sons, 2009.
- Aulas teóricas e práticas acompanhadas pelo Professor Carlos Carvalho, Instituto Superior de Engenharia de Lisboa (ISEL), 2025–2026.